System Design Document (SDD)

System Design Document

Project Title: AI Based Resume Scanner

Version: 1.0

Date: 15/11/2025

Prepared by: [Karan Bodkhe, Atharva

Chirlekar, Rushikesh Aher]

Reviewed by: [Parth kawre]

Approved by: [Sunil Joshi]

System Overview

1. System Description

The **Ai Based Resume Scanner** is a next-generation, AI-driven resume screening platform designed to modernize the hiring process. It is proposed to directly address the failures of slow manual reviews, unconscious human bias, and the rigid, ineffective keyword filtering common in traditional Applicant Tracking Systems (ATS).
The core of the system is a sophisticated engine that leverages **semantic understanding** and **contextual matching** to understand the meaning and intent of a resume, not just the exact keywords.

Main Features:

- **Semantic Matching Engine:** Utilizes state-of-the-art Transformer architectures, specifically **Sentence-BERT**, to deploy a matching engine that understands the meaning and context of a candidate's profile against a job description.
- **Multi-Dimensional Scoring:** A customizable algorithm evaluates candidates on various factors, including skills, experience, education, and resume quality. Recruiters can configure the weights of these factors according to specific job priorities.
- **Blind Screening Mode:** Implements **Named Entity Recognition (NER)** to redact personal identifiers, systematically reducing the potential for unconscious human bias in the early screening stages.
- **Recruiter Dashboard:** An interactive front-end interface that ranks candidates and delivers strategic insights.

2. System Objectives

The primary goals (GENERAL GOALS) of the Ai Based Resume Scanner system are to:

- **Data Pipeline Engineering:** Engineer a robust data pipeline capable of ingesting resumes in multiple formats and converting them into high-quality, machine-readable data.
- **Advanced Semantic Matching:** Deploy a state-of-the-art semantic matching engine to understand meaning rather than just words.
- **Customizable Scoring:** Design a multi-dimensional scoring algorithm that evaluates candidates based on skills, experience, education, and resume quality, while allowing recruiters to configure factor weights.

- **Bias Reduction:** Build a "Blind Screening Mode" that redacts personal identifiers to systematically reduce unconscious bias in the process.
- **Strategic Insights:** Deliver an interactive recruiter dashboard that surfaces strategic insights about applicant pools, skills gaps, and sourcing quality, moving beyond simple candidate rankings.

3. System Environment

Category	Component/Description	Technologies Mentioned
Logic/AI	Core processing and data	Python, Natural Language
Category	Component/Description	Technologies Mentioned
	analysis layer	Processing (NLP), Transformer-based models (Sentence-BERT), Named Entity Recognition (NER)
Application (Backend)	The web server and API layer orchestrating the system	Flask (Python Web Framework), REST APIs
Database	Stores job data, candidate profiles, and screening results	SQLite (for Development), PostgreSQL (for Production)
Interface/Network	User interaction and deployment	Frontend recruiter interface/analytics dashboard, Network environment (standard client-server architecture)

System Architecture Design

The system's architecture for the "AI Based Resume Scanner" is designed as a **three-tier model** to ensure modularity, scalability, and robust performance. The core interactions are managed via a **REST API**.

1. **Presentation Layer (Frontend)**

This is the user interface where recruiters interact with the system.

- **Component:** Recruiter Dashboard.
- **Technologies:** Web-based interface using **HTML, CSS, and JavaScript**.
- **Key Functions:**

- Uploading resumes.
- Managing jobs.
- Visualizing **ranked results**.
- Initiating **API Calls** to the Logic Layer

2. Logic Layer (Backend Engine)

This is the central processing unit, implemented as a service-oriented engine.

- **Component: Flask-powered engine in Python**.
- **Interface: HTTP/HTTPS (REST API)**.
- **Core Modules and Processes:**

  - **Text Extraction:** Handles resume ingestion and preprocessing.
  - **Semantic Embedding:** Uses **Sentence-BERT** (a Transformer-based model) to move beyond keyword searches and transform text into high-dimensional embeddings for true semantic understanding and contextual matching.
  - **Scoring Algorithm:** A multi-dimensional process that combines semantic similarity, experience, education, and resume quality to produce a final, recruiter-configurable score.
  - **Bias Reduction:** Implements a Blind Screening Mode using **spaCy's Named Entity Recognition (NER)** and **regex-based detection** to redact sensitive identifiers.
  - **Orchestration:** Orchestrates data ingestion, semantic analysis, scoring, and API delivery
  - .

4. Data Layer

This layer provides persistent storage for data, configurations, and files.

- **Components:** Database and Secure File Storage.
- **Technologies:**

  - **Database: SQLite** for development and **PostgreSQL** for production.
  - **Interactions:** The Logic Layer communicates using **ORM/SQL Queries** and **File I/O (SQL)**.
  -
- **Data Stored:** Jobs, Candidates, Screening Results, and flexible JSON support for scoring weights and analytics. Resumes are stored in **Secure File Storage**.

5. Analytics Module

This is a key output component that delivers strategic insights.

- **Data Flow:** Receives the **Analytics Data Flow** from the Logic Layer.

- **Insights Provided:**
    - **Skill Gaps**.
    - **Source Insights** (sourcing trends).
    - Ranked Results Dashboard.

Architectural Design Overview:

The "AI Based Resume Scanner" is engineered using a **Layered Architecture**, often referred to as a **Three-Tier Model**. This architectural pattern is selected to enforce a clear separation of concerns, thereby maximizing **modularity, scalability, and maintainability**. The system is logically partitioned into the following three distinct tiers, with all interaction between the top and middle tiers managed via a robust **REST API**:

1. **Presentation Layer (Recruiter Dashboard)**

This tier constitutes the client-side interface, focusing exclusively on user interaction.

- **Role:** Provides the interface for recruiters to upload resumes, manage job postings, and visualize candidate rankings and strategic analytics.
- **Technology Stack:** Built using standard web technologies: **HTML, CSS, and JavaScript**.
- **Interactions:** Communicates with the Logic Layer using **API Calls** over HTTP/HTTPS.

2. **Logic Layer (Flask Backend)**

This is the core functional tier that contains all the business logic, intelligent processing, and orchestration.

- **Role:** Serves as the **Flask-powered engine in Python** responsible for data ingestion, semantic analysis, scoring, and API delivery.

- **Key Components**
    - **Semantic Intelligence:** Utilizes **Transformer-based models (Sentence-BERT)** to move beyond brittle keyword searches into true contextual and semantic understanding.
    - **Bias Mitigation:** Executes the **Blind Screening Mode** via **Named Entity Recognition (NER)**, augmented with regex, to redact sensitive identifiers.
    - **Scoring Engine:** Runs the multi-dimensional algorithm that combines various factors (skills, experience, education, quality) into a final score.

3. Data Layer (Persistence and Storage)

This foundational tier ensures secure and persistent data management.

- **Role:** Stores all persistent application data, configurations, and source documents.

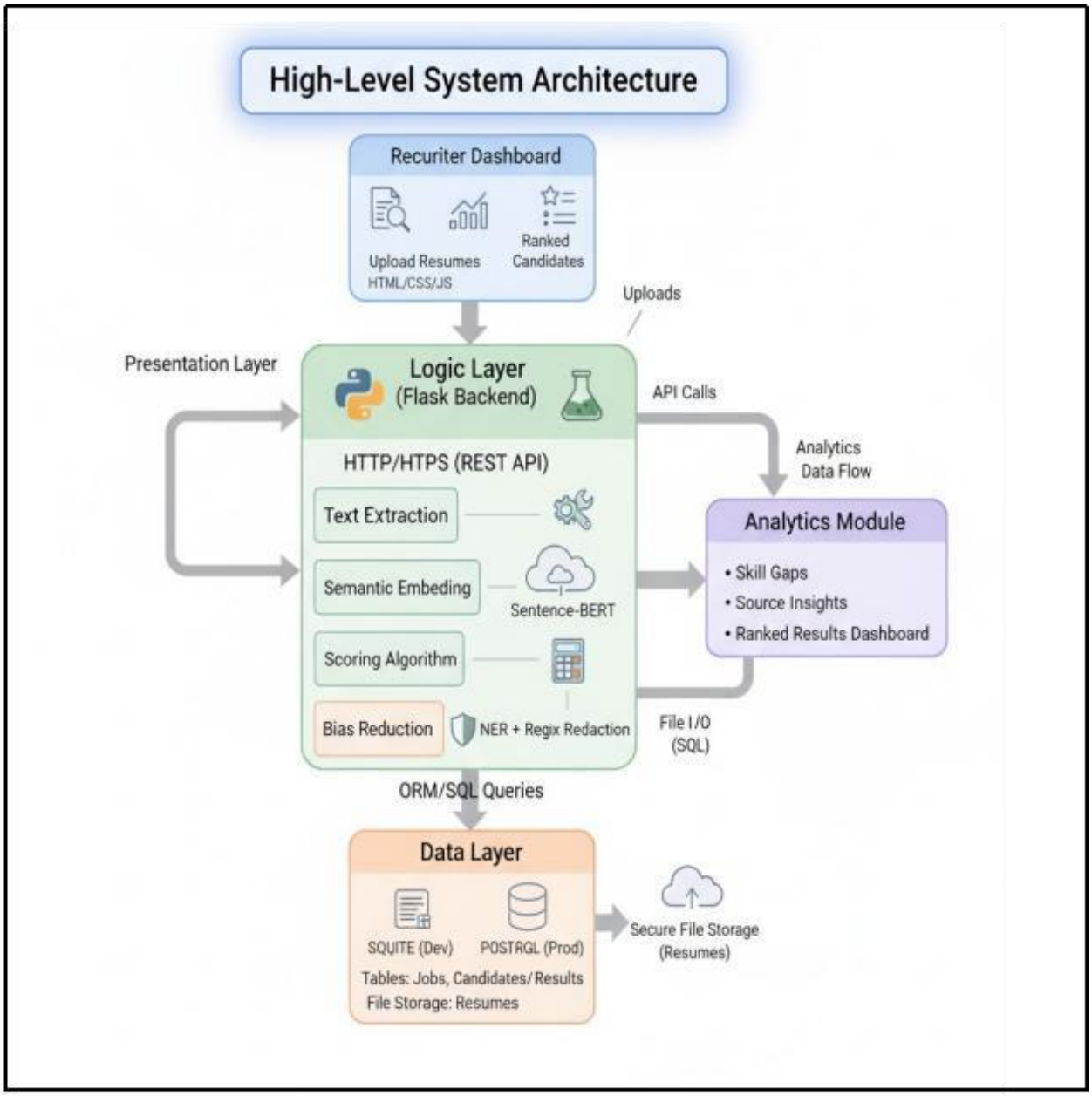
- **Technology Stack:**

    - **Database: SQLite** for development and **PostgreSQL** for production, supporting flexible scoring weights via JSON.
    - **Interaction:** Managed via **ORM/SQL Queries** from the Logic Layer.

- **Storage:** Includes secure file storage for the source resumes.

This layered structure facilitates independent development and scaling of each tier, allowing the Logic Layer to evolve with more sophisticated AI models without impacting the Presentation or Data Layer

System Architecture Diagram



The architecture follows a **Three-Tier Model** with a clear separation of concerns between the Presentation, Application (Logic), and Data layers.

1. Presentation Tier (Client)

- **Component:** Recruiter Dashboard (built with HTML, CSS, JavaScript).
- **Role:** The entry point for the user. Recruiters interact here to upload job descriptions and resumes, configure the multi-dimensional scoring weights, and view results.
- **Data Flow:** Initiates all requests to the Backend via **REST APIs** (e.g., submitting a job, fetching candidate rankings).

2. Application Tier (Backend/Logic Layer)

- **Component:** Core Engine (Built with **Python** and **Flask**).
- **Role:** The core intelligence and processing unit. It handles all business logic, data processing, and AI execution.
- **Key Modules & Flow (Internal):**
    1. **Ingestion/Parsing Module:** Receives raw resumes and job descriptions from the Presentation Tier and extracts structured data (using techniques like Named Entity Recognition or regular expressions).
    2. **Bias Mitigation Module:** Runs the extracted text through **spaCy's NER** and regex to redact personal identifiers, creating a *Blind Profile*.
    3. **Semantic Analysis Module:** Uses **Sentence-BERT** (a Transformer model) to convert both the job description and the candidate's core profile into numerical vector embeddings, allowing for contextual similarity calculation.
    4. **Contextual Scoring Module:** Applies the multi-dimensional scoring algorithm, using the recruiter-configured weights and the semantic similarity score to generate a final **Match Score** and a **Ranking**.
    5. **API Handler:** Manages communication, sending results and insights back to the Presentation Tier and managing data persistence in the Data Tier.

3. Data Tier (Persistence Layer)

- **Component:** Database (SQLite for Development, **PostgreSQL** for Production).
- **Role:** Securely stores all persistent system data.
- **Stored Data:**
    - Job Requirements (Job Descriptions, Required Skills, Configured Scoring Weights).
    - Candidate Profiles (Parsed, structured data from resumes).
    - Screening Results (Match Scores, Rankings, Historical Data for trends).

Data Flow Summary (High-Level)

1. **(Input):** Recruiter uploads Job Description and Resumes via **Recruiter Dashboard**.
2. **(Processing Step 1 - Ingestion):** Dashboard sends files to **Core Engine** via REST API.
3. **(Processing Step 2 - Analysis):** Core Engine processes raw files through **Parsing, Bias Mitigation, and Semantic Analysis**.
4. **(Processing Step 3 - Scoring):** Core Engine applies the **Contextual Scoring Algorithm** to generate the candidate rank/score.
5. **(Storage):** Core Engine stores the structured Candidate Profiles and the final Match Scores in the **Database**.
6. **(Output):** Core Engine retrieves the final, ranked list and strategic insights from the Database and sends it back to the **Recruiter Dashboard** for display.

Component Description

Component Name	Description	Technologies Used
Recruiter Dashboard (Presentation Tier)	The user-facing interface for the system. It handles user input (uploading job descriptions and resumes) and visual output (displaying candidate rankings, scores, and strategic insights). It communicates with the Core Engine via REST APIs.	HTML, CSS, JavaScript

Core Engine (Application Tier)	The central backend logic layer built on the Flask framework. It orchestrates all processes, managing data flow between the dashboard, the AI modules, and the database. It exposes the necessary **REST APIs** to the frontend.	Python, **Flask** (Web Framework)
Ingestion & Parsing Module	Responsible for extracting text and key data (skills, experience, education, contact info) from unstructured resume and job description files (e.g., DOCX, PDF) and converting them into a structured, machine-readable format (e.g., JSON).	Python libraries (likely python-docx, PyMuPDF, or similar), Regex
Semantic Analysis Module	The core intelligence component that moves beyond keyword matching. It uses deep learning models to understand the **context, meaning, and intent** of the text, calculating the true similarity between a resume and a job description.	**Sentence-BERT** (Transformer-based Model), **Natural Language Processing (NLP)**
Bias Mitigation Module	Executes the "Blind Screening Mode." It identifies and redacts specific personal identifiers (names, emails, gendered pronouns, etc.) from the candidate profiles *before* the main scoring, ensuring	**spaCy** (Named Entity Recognition - NER), Regex

Contextual Scoring Module	A customizable algorithm that generates the final match score for each candidate. It combines the semantic similarity score with weighted factors like skills, experience, and resume quality, allowing recruiters to **configure the weights**.	Python
Database Layer	The persistent storage mechanism for all system data. It stores structured candidate profiles, job descriptions, configuration settings (like scoring weights), and the final screening results/rankings.	**SQLite** (Development), **PostgreSQL** (Production)

Module Overview: AI Based Resume Scanner

The AI Based Resume Scanner is organized into a **three-tier architecture** consisting of a Presentation Layer, a Logic Layer (Flask Backend), and a Data Layer. This modular approach emphasizes scalability, fairness, and robust software design.

1. Presentation Layer (Recruiter Dashboard)

4. **Role:** Serves as the **human-centric interface** for recruiters.
5. **Key Functions:**
    1. Allows recruiters to **upload resumes** and manage job postings.
    2. Displays **ranked candidates** lists.
    3. Provides a **visual dashboard** for analytics.
6. **Technology:** Built using standard **HTML, CSS, and JavaScript**.

2. Logic Layer (Flask Backend)

4. **Role:** The **core engine** that orchestrates the entire process, including data ingestion, AI analysis, scoring, and API delivery. It uses a **Python- based Flask engine**.
5. **Key Subsystems:**

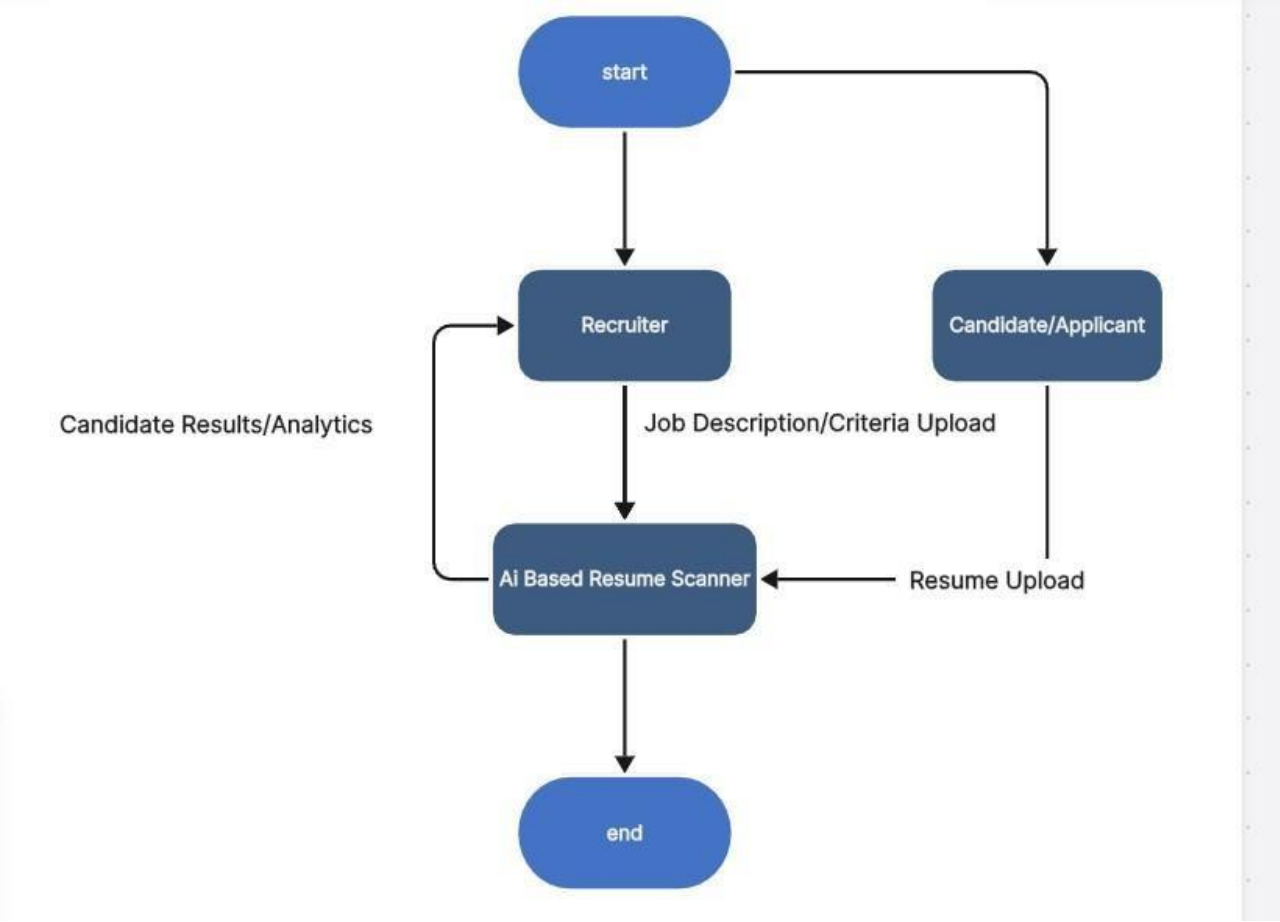
Subsystem	Core Role	Technology / Approach
Text Extraction	Ingests resumes (PDF/DOCX) and converts them into structured, machine- readable text.	Custom parsing pipeline.
Semantic Embedding	Transforms job descriptions and resumes into **high- dimensional embeddings** to calculate contextual relevance.	**Transformer architectures** (specifically **Sentence- BERT**).
Scoring Algorithm	Calculates the final score by combining semantic similarity, experience, education, and resume quality.	Multi-dimensional, recruiter-configurable algorithm.
Bias Reduction	Redacts sensitive personal identifiers to ensure evaluations are based on merit and minimize unconscious bias.	**Named Entity Recognition (NER)** (using **spaCy**) augmented with **regex- based detection**.
Analytics Module	Provides **strategic insights** to HR professionals.	Surfaces data on **Skill Gaps**, **Sourcing Insights**, and recruitment bottlenecks.
REST API	Handles communication between the Presentation and Data Layers via **HTTP/HTTPS**.	Flask-designed APIs.

3. Data Layer

- **Role:** Manages the secure storage and retrieval of all persistent data.
- **Key Functions:**
  - Stores tables for **Jobs, Candidates, and Screening Results**.
  - Provides file storage for the raw resumes.
- **Technology:** Uses **SQLite** for development and **PostgreSQL** for production environments, leveraging SQL for queries and JSON support

for flexible scoring weights.

Data Flow Diagram



Recruiter Dashboard Interface (Frontend)

The Frontend is a web-based interface built with **HTML, CSS, and JavaScript**. It serves as the single point of interaction for the Recruiter.

Key Views and Components

1. Job Management View

- **Purpose:** To create, manage, and define the screening criteria for new job postings.
- **Interactions:**
  - **New Job Creation:** Recruiter inputs the **Job Title** and uploads the full **Job Description Text**.
  - **Scoring Configuration:** A section allows the recruiter to adjust the **Scoring Weights** (e.g., set experience importance higher than education).
  - **Resume Upload:** A drag-and-drop or file-picker interface for uploading resumes in **PDF or DOCX** formats.

2. Candidate Ranking View (Core Output)

- **Purpose:** To display and interact with the results of the AI screening process.
- **Components:**
  - **Ranked List:** Candidates are displayed in descending order based on the **Final Score**.
  - **Score Breakdown:** Upon clicking a candidate, a sidebar or modal reveals the **multi-dimensional scores** (e.g., Semantic Similarity, Experience Score, Resume Quality Score).
  - **Blind Screening Toggle:** A visible switch to enable/disable the **Blind Screening Mode**. When active, the candidate's name, email, and other identifiers are redacted.

3. Analytics Dashboard View

- **Purpose:** To surface strategic insights about the applicant pool.
- **Components:**
  - **Skill Gap Analysis:** Visualizations (e.g., bar charts) showing the most common and most missing skills among the applicant pool relative to the job requirements.
  - **Sourcing Trends:** Charts showing the quality or volume of candidates from different application sources.
  - **Recruitment Bottlenecks:** Data visualizations to help identify trends and issues in the hiring process.

API Endpoints (Backend REST API)

The Logic Layer (**Flask Backend**) exposes a **REST API** over **HTTP/HTTPS** to handle all data and process orchestration.

Endpoint Category	Method	Endpoint Path	Description

Endpoint Category	Method	Endpoint Path	Description
Job Management	POST	/api/v1/jobs	Creates a new job posting and stores the job description.
Job Management	PUT	/api/v1/jobs/{job_id}	Updates job scoring weights or status.
Resume Ingestion	POST	/api/v1/jobs/{job_id} /resumes/upload	Receives and processes resume files (PDF/DOCX) for a specific job.
Screening Results	GET	/api/v1/jobs/{job_id} /results	Retrieves the ranked list of candidates and their final scores for a job.
Analytics	GET	/api/v1/jobs/{job_id} /analytics	Fetches aggregated data for Skill Gap and Source Insight analysis.

Interactions with External Systems

The system architecture defines two main interactions outside of the core Flask application:

1. Data Layer Interaction (Internal)

- **System: Database Layer** (SQLite/PostgreSQL).
- **Interaction:** The Logic Layer communicates with the Database using **ORM/SQL Queries** to store and retrieve:
  - Job descriptions and scoring weights.
  - Processed/redacted candidate data and semantic embeddings.
  - The final screening results and analytics data.
  - Resumes are also securely stored via **File I/O** separate from the database tables.

2. Recruiter (External Entity)

- **System:** The **Recruiter** (User).
- **Interaction:** The Recruiter submits inputs (resumes, job data) and receives outputs (ranked candidates, dashboard analytics) via the **Presentation Layer**, which communicates through the **REST API**.

Wireframe Mockup: Candidate Ranking View

This conceptual wireframe illustrates the key elements of the candidate ranking view within the Recruiter Dashboard.